

APOSTILA

Desenvolvedor Python

Módulo Específico I – Fundamentos 2

Estruturas de Dados e Funções

Curso Técnico em Desenvolvimento de Sistemas
2026

Apresentação

Você já sabe criar programas básicos em Python, trabalhar com variáveis, tipos de dados, operadores, condicionais, laços e funções simples. Agora é hora de dar um salto de qualidade: aprender a organizar e manipular dados de forma eficiente e a construir funções mais poderosas e reutilizáveis.

Ao final você será capaz de usar com segurança as principais estruturas de dados do Python — listas, tuplas, dicionários e conjuntos — e de criar funções bem projetadas com diferentes tipos de parâmetros e retorno.

Aula	Tema	Carga
Aula 1	Listas — fundo do poço	4h
Aula 2	Tuplas e Conjuntos	4h
Aula 3	Dicionários	4h
Aula 4	Funções avançadas	4h
Aula 5	Compreensões e funções built-in	4h
Aula 6	Projeto integrador	4h

1 – Listas: Estrutura, Métodos e Fatiamento

1.1 Revisão rápida e por que ir além

Na UC1 você conheceu listas de forma introdutória. Agora vamos fundo: listas são a estrutura de dados mais versátil do Python. Quase tudo que envolve coleções de valores passa por elas.

Metáfora: A planilha de um professor

Imagine uma lista como a coluna de notas numa planilha:

- Cada célula tem uma posição (índice)
- Você pode inserir, apagar, reordenar
- Pode percorrer a coluna inteira de uma vez
- A planilha não tem tamanho fixo — cresce conforme necessário

1.2 Criação e acesso

```
# Criando listas de diferentes formas
notas    = [8.5, 7.0, 9.2, 6.8, 10.0]
nomes    = ['Ana', 'Bruno', 'Carlos', 'Diana']
mista    = [1, 'dois', 3.0, True]           # tipos mistos: ok
vazia    = []                             # lista vazia
repetida = [0] * 5                         # [0, 0, 0, 0, 0]
```

```
# Acesso por índice (começa em 0)
print(notas[0])    # 8.5 - primeiro
print(notas[-1])   # 10.0 - último
print(notas[-2])   # 6.8 - penúltimo

# Modificando um elemento
notas[1] = 7.5
print(notas)      # [8.5, 7.5, 9.2, 6.8, 10.0]
```

1.3 Fatiamento (slicing)

Fatiamento permite extrair partes de uma lista usando a sintaxe `lista[início:fim:passo]`. O índice fim não é incluído.

```
letras = ['a','b','c','d','e','f','g']

print(letras[1:4])    # ['b','c','d'] - índices 1,2,3
print(letras[:3])     # ['a','b','c'] - do início até 2
print(letras[4:])     # ['e','f','g'] - do 4 até o fim
print(letras[::2])    # ['a','c','e','g'] - de 2 em 2
print(letras[::-1])   # ['g','f','e','d','c','b','a'] - invertida

# Fatiamento não modifica a original - cria uma nova lista
primeiros = letras[:3]
print(primeiros)      # ['a','b','c']
print(letras)         # ['a','b','c','d','e','f','g'] - intacta
```

1.4 Métodos essenciais

Método	O que faz	Exemplo
<code>append(x)</code>	Adiciona x ao final	<code>lista.append(10)</code>
<code>insert(i,x)</code>	Insere x na posição i	<code>lista.insert(0,'início')</code>
<code>remove(x)</code>	Remove a 1ª ocorrência de x	<code>lista.remove('Ana')</code>
<code>pop(i)</code>	Remove e retorna o elemento da posição i	<code>lista.pop(-1)</code>
<code>index(x)</code>	Retorna o índice de x	<code>lista.index('Bruno')</code>
<code>count(x)</code>	Conta ocorrências de x	<code>lista.count(7.0)</code>
<code>sort()</code>	Ordena no lugar (modifica)	<code>lista.sort()</code>
<code>sorted()</code>	Retorna nova lista ordenada	<code>nova = sorted(lista)</code>
<code>reverse()</code>	Inverte no lugar	<code>lista.reverse()</code>
<code>len()</code>	Tamanho da lista	<code>len(lista)</code>
<code>in</code>	Verifica se existe	<code>'Ana' in lista</code>

```
alunos = ['Carlos', 'Ana', 'Diana', 'Bruno']

alunos.append('Eduardo')      # adiciona no fim
alunos.insert(0, 'Alice')     # insere na posição 0
alunos.remove('Diana')        # remove por valor
ultimo = alunos.pop()         # remove e retorna o último

print(alunos)    # ['Alice', 'Carlos', 'Ana', 'Bruno']
print(ultimo)    # Eduardo

alunos.sort()    # ordena em ordem alfabética
print(alunos)    # ['Alice', 'Ana', 'Bruno', 'Carlos']

print('Ana' in alunos)    # True
print(alunos.index('Bruno'))    # 2
```

1.5 Listas aninhadas (matriz)

Uma lista pode conter outras listas, formando estruturas bidimensionais como matrizes ou tabelas.

```
# Matriz 3x3
matriz = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

print(matriz[0])    # [1, 2, 3] - primeira linha
print(matriz[1][2]) # 6         - linha 1, coluna 2

# Percorrendo a matriz
for linha in matriz:
    for elemento in linha:
        print(elemento, end=' ')
    print() # quebra de linha após cada linha da matriz
```

1.6 Percorrendo listas com eficiência

```
frutas = ['maçã', 'banana', 'laranja']

# Forma simples
for fruta in frutas:
    print(fruta)

# Com índice usando enumerate()
for i, fruta in enumerate(frutas):
    print(f'{i}: {fruta}')
# 0: maçã 1: banana 2: laranja
```

```
# Percorrendo duas listas juntas com zip()
nomes = ['Ana', 'Bruno', 'Carlos']
notas = [9.0, 7.5, 8.2]

for nome, nota in zip(nomes, notas):
    print(f'{nome}: {nota}')
```

2 – Tuplas e Conjuntos

2.1 Tuplas — a lista imutável

Uma tupla é como uma lista, mas seus elementos não podem ser alterados após a criação. Usa parênteses () em vez de colchetes [].

Metáfora: O registro oficial

Pense numa tupla como um documento oficial carimbado:

- Uma vez emitido, não se pode alterar
- Você pode ler qualquer campo, mas não editar
- Ideal para dados que não devem mudar: coordenadas, cores RGB, datas de nascimento

```
# Criando tuplas
coordenada = (-15.77, -47.92)          # latitude, longitude Brasília
rgb_vermelho = (255, 0, 0)
dias_semana = ('seg', 'ter', 'qua', 'qui', 'sex', 'sáb', 'dom')
unitaria = (42,)                      # vírgula obrigatória para
tupla de 1 elemento

# Acesso igual a listas
print(coordenada[0])  # -15.77
print(dias_semana[-1]) # dom

# Tentativa de modificar – gera erro!
# coordenada[0] = 0  # TypeError: 'tuple' object does not support item
assignment

# Desempacotamento (unpacking)
lat, lon = coordenada
print(f'Latitude: {lat} Longitude: {lon}')
```

```
# Contagem e busca
print(dias_semana.count('seg'))  # 1
print(dias_semana.index('qua'))  # 2
```

Quando usar tupla vs lista?

Característica	Lista []	Tupla ()
Mutabilidade	Mutável (pode alterar)	Imutável (fixo)
Performance	Levemente mais lenta	Levemente mais rápida
Uso típico	Coleções que mudam	Dados fixos, coordenadas, retorno múltiplo
Pode ser chave de dict?	Não	Sim

Retornando múltiplos valores com tupla

```
def minmax(lista):
    return min(lista), max(lista)    # retorna uma tupla

notas = [7.5, 9.0, 5.5, 8.2, 6.0]
menor, maior = minmax(notas)        # desempacota
print(f'Menor: {menor} Maior: {maior}')
```

2.2 Conjuntos (set)

Um conjunto é uma coleção não ordenada de elementos únicos — sem repetições e sem índice.

🎯 Metáfora: A lista de chamada sem duplicatas

Se você chamar 'Ana' duas vezes na chamada, ela é registrada uma só vez.

Conjuntos funcionam assim: cada elemento aparece apenas uma vez, não importa quantas vezes você tente adicionar.

```
# Criando conjuntos
cores = {'vermelho', 'azul', 'verde'}
numeros = {1, 2, 3, 2, 1}    # duplicatas ignoradas
print(numeros)              # {1, 2, 3}

# Criando a partir de uma lista (remove duplicatas!)
lista = [1, 2, 2, 3, 3, 3, 4]
unico = set(lista)
print(unico)                # {1, 2, 3, 4}

# Operações fundamentais
cores.add('amarelo')        # adiciona
cores.discard('azul')       # remove sem erro se não existir
print('verde' in cores)     # True - verificação rápida

# Operações de teoria dos conjuntos
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
```

```
print(a | b)    # União:      {1,2,3,4,5,6}
print(a & b)    # Interseção: {3,4}
print(a - b)    # Diferença:  {1,2}
print(a ^ b)    # Dif. simét.: {1,2,5,6}
```

Caso de uso clássico: remover duplicatas de uma lista

```
# Problema: lista de inscrições com duplicatas
inscricoes = ['Ana', 'Bruno', 'Ana', 'Carlos', 'Bruno', 'Diana']

# Solução elegante com set
unicos = list(set(inscricoes))
print(unicos)          # ordem não garantida

# Se precisar manter a ordem de inserção:
sem_dup = list(dict.fromkeys(inscricoes))
print(sem_dup)         # ['Ana', 'Bruno', 'Carlos', 'Diana']
```

3 – Dicionários

3.1 O que é um dicionário?

Um dicionário armazena pares chave: valor. Em vez de acessar dados por posição numérica, você acessa por uma chave significativa. É a estrutura de dados mais importante do Python depois das listas.

Metáfora: O dicionário real

Num dicionário de português você não busca a palavra pela posição ('palavra número 4721').

Você busca pelo nome da palavra (chave) e encontra o significado (valor).

```
{'python': 'linguagem de programação de alto nível',
 'lista': 'estrutura de dados ordenada e mutável'}
```

3.2 Criação e acesso

```

# Criando um dicionário
aluno = {
    'nome':      'Carlos',
    'idade':     17,
    'curso':     'Desenvolvedor Python',
    'nota':      8.5,
    'aprovado':  True
}

# Acessando valores
print(aluno['nome'])      # Carlos
print(aluno['nota'])      # 8.5

# Forma segura com .get() – sem KeyError se não existir
print(aluno.get('email')) # None
print(aluno.get('email', 'N/A')) # N/A (valor padrão)

# Adicionando e modificando
aluno['email'] = 'carlos@senai.df' # nova chave
aluno['nota'] = 9.0                # atualiza valor existente

# Removendo
del aluno['aprovado']              # remove a chave
nota = aluno.pop('nota')          # remove e retorna
print(nota)                       # 9.0

```

3.3 Métodos essenciais

```

produto = {'nome': 'Notebook', 'preco': 3500.0, 'estoque': 12}

# Iterando
for chave in produto.keys():
    print(chave)

for valor in produto.values():
    print(valor)

for chave, valor in produto.items():
    print(f'{chave}: {valor}')

# Verificando existência de chave
print('preco' in produto) # True
print('desconto' in produto) # False

# Tamanho
print(len(produto)) # 3

# Copiando (nunca use dict2 = dict1 – copia a referência!)
copia = produto.copy()

```


3.4 Dicionários aninhados

Dicionários podem conter outros dicionários como valores, permitindo representar dados hierárquicos.

```
turma = {
    'Ana': {'nota': 9.0, 'frequencia': 90},
    'Bruno': {'nota': 6.5, 'frequencia': 80},
    'Carlos': {'nota': 4.0, 'frequencia': 60},
}

# Acessando dados aninhados
print(turma['Ana']['nota']) # 9.0
print(turma['Carlos']['frequencia']) # 60

# Percorrendo a turma
for nome, dados in turma.items():
    nota = dados['nota']
    freq = dados['frequencia']
    situacao = 'Aprovado' if nota >= 7.0 and freq >= 75 else
'Reprovado'
    print(f'{nome}: {nota} | {freq}% | {situacao}')
```

3.5 Lista de dicionários — padrão muito comum

Na prática, você vai trabalhar frequentemente com listas de dicionários — o equivalente Python a uma tabela de banco de dados.

```
produtos = [
    {'nome': 'Notebook', 'preco': 3500.0, 'categoria': 'eletronico'},
    {'nome': 'Mouse', 'preco': 89.9, 'categoria': 'eletronico'},
    {'nome': 'Cadeira', 'preco': 650.0, 'categoria': 'mobilia'},
    {'nome': 'Monitor', 'preco': 1200.0, 'categoria': 'eletronico'},
]

# Filtrar eletrônicos
eletronicos = [p for p in produtos if p['categoria'] == 'eletronico']
print(len(eletronicos)) # 3

# Ordenar por preço
em_ordem = sorted(produtos, key=lambda p: p['preco'])
for p in em_ordem:
    print(f"{p['nome']:15} R$ {p['preco']:~.2f}")

# Total em estoque
total = sum(p['preco'] for p in produtos)
print(f'Total: R$ {total:~.2f}')
```

4 – Funções Avançadas

4.1 Revisão e motivação

Na UC1 aprendemos funções básicas com parâmetros e retorno simples. Agora vamos expandir: `*args`, `**kwargs`, funções como objetos, recursividade e funções `lambda`.

4.2 Parâmetros `*args` — número variável de argumentos

O `*args` permite que uma função receba qualquer quantidade de argumentos posicionais. Dentro da função, `args` é uma tupla.

```
def somar(*numeros):
    total = 0
    for n in numeros:
        total += n
    return total

print(somar(1, 2))           # 3
print(somar(1, 2, 3, 4, 5)) # 15
print(somar(10))            # 10

# Combinando parâmetro fixo com *args
def apresentar(titulo, *nomes):
    print(f'--- {titulo} ---')
    for nome in nomes:
        print(f'    • {nome}')

apresentar('Turma Python', 'Ana', 'Bruno', 'Carlos', 'Diana')
```

4.3 Parâmetros `**kwargs` — argumentos nomeados

O `**kwargs` permite passar qualquer quantidade de argumentos nomeados. Dentro da função, `kwargs` é um dicionário.

```
def cadastrar_aluno(**dados):
    print('Novo aluno cadastrado:')
    for campo, valor in dados.items():
        print(f'    {campo}: {valor}')

cadastrar_aluno(nome='Ana', idade=17, curso='Python', nota=9.5)

# Combinando tudo
def relatorio(titulo, *itens, **config):
    separador = config.get('separador', '-')
    largura = config.get('largura', 30)
    print(separador * largura)
```

```
print(titulo.center(largura))
print(separador * largura)
for item in itens:
    print(f' {item}')

relatorio('Notas da Turma', 'Ana: 9.0', 'Bruno: 7.5',
          separador=' ', largura=25)
```

4.4 Funções lambda

Lambda cria uma função anônima (sem nome) em uma única linha. Ideal para operações simples usadas pontualmente.

```
# Função normal vs lambda
def dobrar(x):
    return x * 2

dobrar_lambda = lambda x: x * 2

print(dobrar(5))          # 10
print(dobrar_lambda(5))  # 10

# Lambda é muito útil como argumento de sorted(), map(), filter()
alunos = [('Ana', 9.0), ('Carlos', 6.5), ('Bruno', 8.0)]

# Ordenar por nota (segundo elemento da tupla)
por_nota = sorted(alunos, key=lambda a: a[1], reverse=True)
for nome, nota in por_nota:
    print(f'{nome}: {nota}')
```

4.5 Funções como objetos — map() e filter()

```
notas = [4.5, 7.0, 8.5, 3.0, 9.2, 5.5, 7.8]

# map(): aplica função a cada elemento
dobradas = list(map(lambda n: n * 2, notas))
print(dobradas)

# filter(): mantém apenas os que satisfazem a condição
aprovados = list(filter(lambda n: n >= 7.0, notas))
print(aprovados)  # [7.0, 8.5, 9.2, 7.8]

# Equivalente com list comprehension (preferido em Python moderno)
aprovados2 = [n for n in notas if n >= 7.0]
print(aprovados2)  # igual ao anterior
```

4.6 Recursividade

Uma função recursiva é aquela que chama a si mesma. É uma técnica poderosa para resolver problemas que se dividem em subproblemas menores.

Metáfora: As bonecas russas (Matryoshka)

Cada boneca contém uma versão menor de si mesma.

Você abre uma, encontra outra, abre de novo... até chegar à menor (caso base).

Em recursão: cada chamada trabalha um pedaço menor, até chegar ao caso base.

```
# Fatorial com recursão
def fatorial(n):
    if n == 0 or n == 1:    # caso base – para a recursão
        return 1
    return n * fatorial(n - 1) # chamada recursiva

print(fatorial(5))    # 120 → 5 * 4 * 3 * 2 * 1
print(fatorial(0))    # 1

# Como funciona para fatorial(4):
# fatorial(4) = 4 * fatorial(3)
#             = 4 * 3 * fatorial(2)
#             = 4 * 3 * 2 * fatorial(1)
#             = 4 * 3 * 2 * 1 = 24

# Sequência de Fibonacci
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

for i in range(10):
    print(fibonacci(i), end=' ') # 0 1 1 2 3 5 8 13 21 34
```

Cuidado com recursão infinita

Sempre defina um CASO BASE claro (condição de parada).

Sem caso base, a função chama a si mesma indefinidamente → `RecursionError`.

Python tem limite de ~1000 chamadas recursivas por padrão.

5 – Compreensões de Lista e Funções Built-in

5.1 List Comprehension

List comprehension é a forma mais pythônica de criar listas a partir de outras coleções. É mais concisa e geralmente mais rápida que um laço for equivalente.

💡 O jeito Python de fazer

Quando um experiente programador Python vê um laço for que só cria uma lista, ele pensa imediatamente em list comprehension.

Não é obrigatório, mas é considerado boa prática no mundo Python.

```
# Forma tradicional vs comprehension

# Tradicional
quadrados = []
for x in range(1, 6):
    quadrados.append(x ** 2)

# Comprehension – equivalente, mais conciso
quadrados = [x ** 2 for x in range(1, 6)]
print(quadrados)    # [1, 4, 9, 16, 25]

# Com condição (filtro)
pares = [x for x in range(1, 21) if x % 2 == 0]
print(pares)    # [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]

# Com transformação e filtro
notas = [4.5, 7.0, 8.5, 3.0, 9.2, 5.5]
aprovados_dobro = [n * 2 for n in notas if n >= 7.0]
print(aprovados_dobro)    # [14.0, 17.0, 18.4]

# Aninhada (equivale a for duplo)
combinacoes = [(x, y) for x in [1,2,3] for y in ['a','b']]
print(combinacoes)
```

5.2 Dict Comprehension e Set Comprehension

```
# Dict comprehension
nomes = ['Ana', 'Bruno', 'Carlos']
tamanhos = {nome: len(nome) for nome in nomes}
print(tamanhos)    # {'Ana': 3, 'Bruno': 5, 'Carlos': 6}

# Invertendo chaves e valores
original = {'a': 1, 'b': 2, 'c': 3}
invertido = {v: k for k, v in original.items()}
print(invertido)    # {1: 'a', 2: 'b', 3: 'c'}

# Set comprehension
letras = {c.lower() for c in 'Python é Incrível' if c.isalpha()}
print(letras)    # conjunto de letras únicas
```

5.3 Funções built-in para coleções

Função	O que faz	Exemplo
len(x)	Tamanho da coleção	len([1,2,3]) → 3
sum(x)	Soma os elementos	sum([1,2,3]) → 6
min(x)	Menor elemento	min([3,1,2]) → 1
max(x)	Maior elemento	max([3,1,2]) → 3
sorted(x)	Retorna lista ordenada	sorted([3,1,2]) → [1,2,3]
reversed(x)	Iterador invertido	list(reversed([1,2,3]))
enumerate(x)	Pares (índice,valor)	for i,v in enumerate(lista)
zip(x,y)	Combina duas coleções	for a,b in zip(l1,l2)
any(x)	True se algum for True	any([False,True,False])
all(x)	True se todos forem True	all([True,True,True])

```

notas = [8.5, 7.0, 4.5, 9.2, 6.0]

print(sum(notas) / len(notas))      # média: 7.04
print(any(n < 5.0 for n in notas))  # True (tem reprovado)
print(all(n >= 7.0 for n in notas)) # False (nem todos aprovados)

# sorted com key
palavras = ['banana', 'maçã', 'kiwi', 'abacate']
por_tamanho = sorted(palavras, key=len)
print(por_tamanho)  # ['kiwi', 'maçã', 'banana', 'abacate']

# zip para combinar listas
nomes = ['Ana', 'Bruno', 'Carlos']
notas_p1 = [8.0, 7.5, 9.0]
notas_p2 = [7.0, 8.0, 6.5]

resultados = [(n, (p1+p2)/2) for n, p1, p2 in zip(nomes, notas_p1,
notas_p2)]
for nome, media in resultados:
    print(f'{nome}: {media:.1f}')

```

5.4 Strings como coleções — métodos importantes

Strings em Python são imutáveis mas têm muitos métodos úteis e podem ser percorridas como listas.

```

texto = '  Olá, Mundo! Python é Incrível.  '

print(texto.strip())      # remove espaços nas bordas
print(texto.lower())      # tudo minúsculo
print(texto.upper())      # tudo maiúsculo
print(texto.title())      # Primeira Letra Maiúscula

```

```
print(texto.replace('!', '.')) # substitui caracteres
print(texto.count('é'))        # conta ocorrências
print('Python' in texto)       # True – busca substring

# split: divide string em lista
frase = 'Python é muito legal'
palavras = frase.split()      # divide por espaços
print(palavras) # ['Python', 'é', 'muito', 'legal']

# join: une lista em string
unida = ' | '.join(palavras)
print(unida)    # Python | é | muito | legal

# f-string com formatação avançada
valor = 1234567.89
print(f'{valor:,.2f}')        # 1,234,567.89
print(f'{valor:>15.2f}')      # alinhado à direita em 15 chars
```

Referências

- Documentação oficial Python: <https://docs.python.org/pt-br/3/>
- Real Python – Estruturas de dados: <https://realpython.com/python-data-structures/>
- PEP 8: <https://pep8.org/>
- Python Crash Course – Eric Matthes (No Starch Press)
- Python Fluente – Luciano Ramalho (Novatec)